8. MISSIONARIES CANNIBAL PROBLEM

Aim:

To solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm in Artificial Intelligence.

Objective :

To safely transport three missionaries and three cannibals from the left river bank to the right river bank using a boat that can carry a maximum of two people, without ever allowing the cannibals to outnumber the missionaries on either bank.

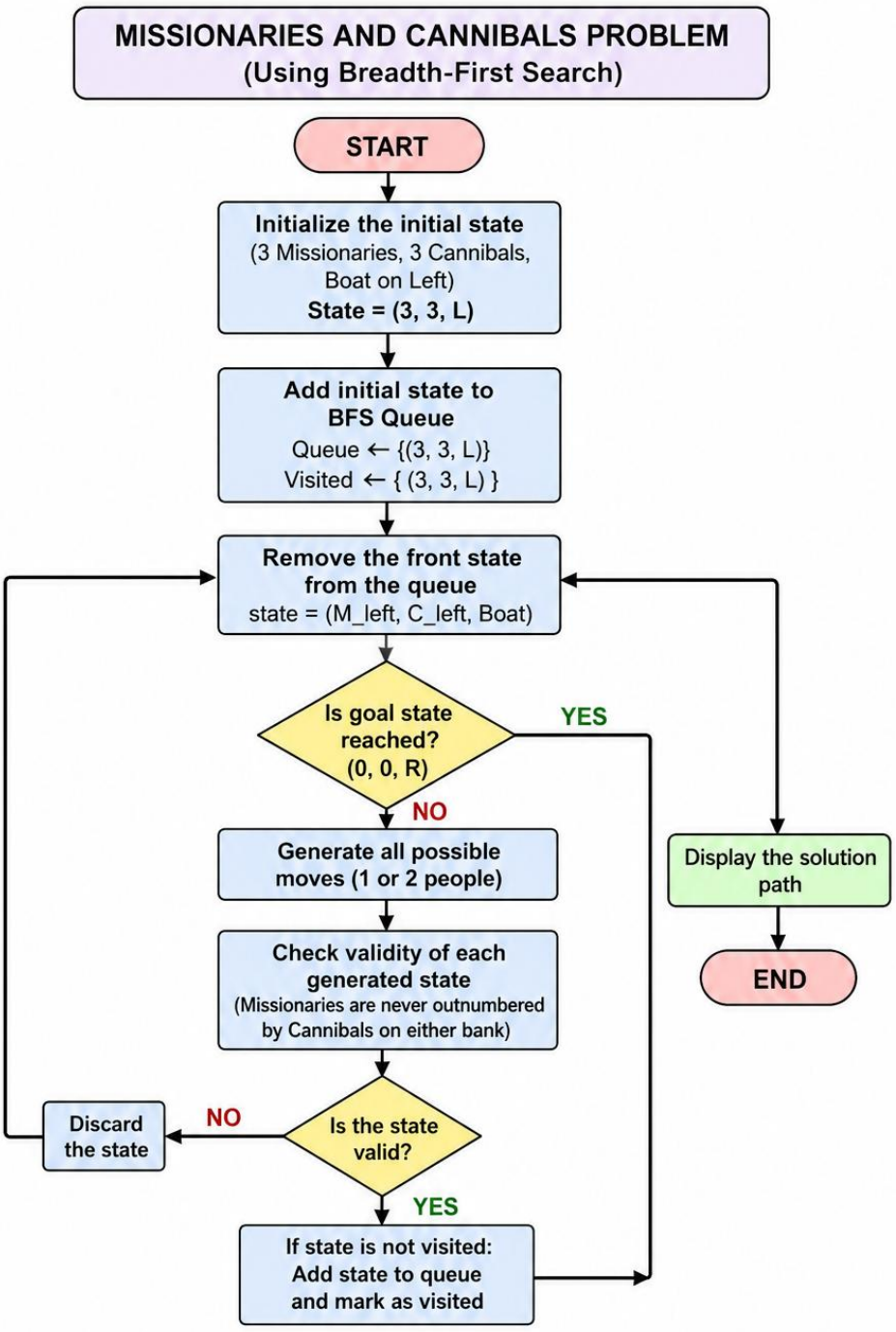
Algorithm:

1. Represent each state as **(Missionaries Left, Cannibals Left, Boat Position)**.
2. Initialize the starting state as **(3,3,L)**.
3. Define the goal state as **(0,0,R)**.
4. Generate all possible boat moves:

- Two missionaries
- Two cannibals
- One missionary and one cannibal
- One missionary
- One cannibal

5. Check whether each generated state is valid.
6. Use Breadth-First Search (BFS) to explore all valid states.
7. Avoid revisiting previously explored states.
8. Continue until the goal state is reached.
9. Display the solution path.

Flowchart :


MISSIONARIES AND CANNIBALS PROBLEM
(Using Breadth-First Search)
START
Initialize the initial state
(3 Missionaries, 3 Cannibals, Boat on Left)
State = (3, 3, L)
Add initial state to BFS Queue
Queue ← {(3, 3, L)}
Visited ← { (3, 3, L) }
Remove the front state from the queue
state = (M_left, C_left, Boat)
Is goal state reached? (0, 0, R)
YES
NO
Display the solution path
END
Generate all possible moves (1 or 2 people)
Check validity of each generated state
(Missionaries are never outnumbered by Cannibals on either bank)
Is the state valid?
NO
Discard the state
YES
If state is not visited: Add state to queue and mark as visited

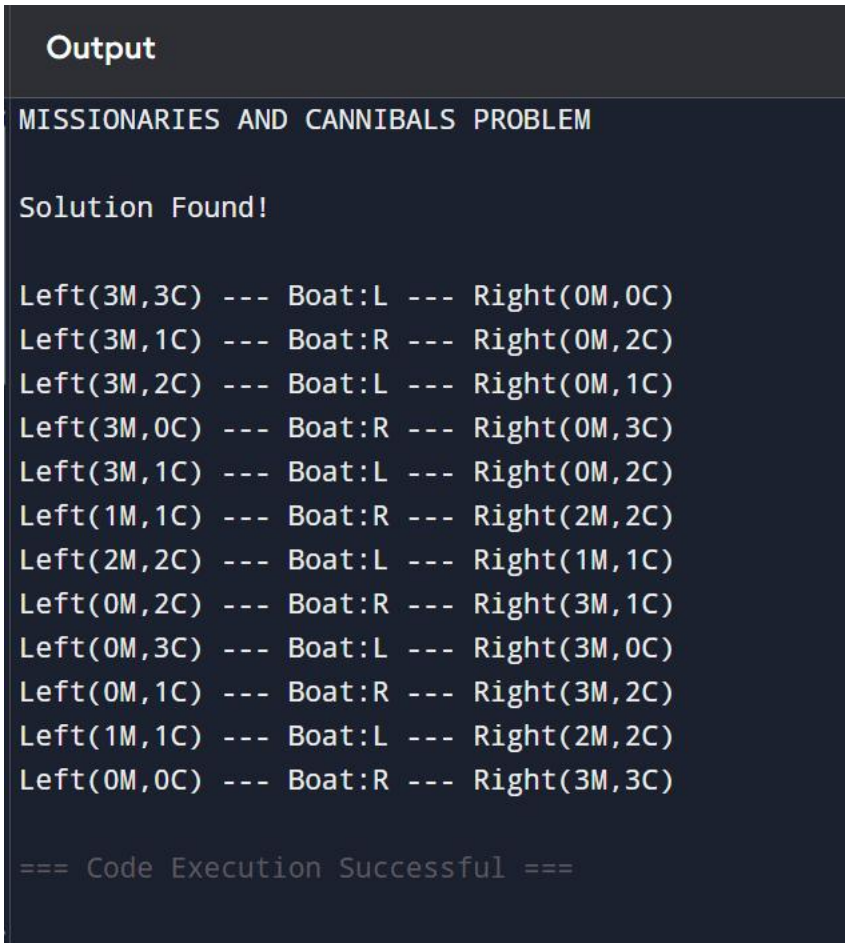
Code:

```
from collections import deque
def is_valid(m_left, c_left, m_right, c_right):
    if m_left < 0 or c_left < 0 or m_right < 0 or c_right < 0:
        return False
    if (m_left > 0 and m_left < c_left):
        return False
    if (m_right > 0 and m_right < c_right):
        return False
    return True
def get_next_states(state):
    m_left, c_left, boat = state
    next_states = []
    moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]
    for m, c in moves:
        if boat == 'L':
            new_state = (
                m_left - m,
                c_left - c,
                'R'
            )
        else:
            new_state = (
                m_left + m,
```

```
                c_left + c,
                'L'
            )
        new_m_left, new_c_left, new_boat = new_state
        new_m_right = 3 - new_m_left
        new_c_right = 3 - new_c_left
        if is_valid(new_m_left, new_c_left, new_m_right, new_c_right):
            next_states.append(new_state)
    return next_states
def bfs():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')
    queue = deque([(start, [start])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state in visited:
            continue
        visited.add(state)
        if state == goal:
            print("\nSolution Found!\n")
            for step in path:
                ml, cl, boat = step
                mr = 3 - ml
                cr = 3 - cl
                print(f"Left({ml}M,{cl}C) --- Boat:{boat} --- Right({mr}M,{cr}C)")
```

```
            return
        for next_state in get_next_states(state):
            if next_state not in visited:
                queue.append((next_state, path + [next_state]))
    print("No Solution Found.")
print("MISSIONARIES AND CANNIBALS PROBLEM")
bfs()
```

Output:



Result :

The Missionaries and Cannibals Problem was successfully solved using the Breadth-First Search (BFS) algorithm. The algorithm explored valid states systematically and found a safe sequence of moves to transport all missionaries and cannibals across the river without violating the problem constraints.